EXPLORATORY DATA ANALYSIS

CHAPTER 03

Data Quality Assessment

Determine what can be trusted, what needs investigation, and what limitations must be communicated.

A practical, concept-first course for quantitative and visual EDA

Learning objectives

Assess completeness, validity, consistency, uniqueness, accuracy, and timeliness; quantify missingness; investigate duplicate records and keys; check domain and cross-column constraints; interpret quality metrics; and communicate findings without overstating what automated checks prove.

1. Review of Chapter 1 — What Is EDA?

EDA is an iterative investigation before analytical or modeling decisions. Its core questions are: What do I have? Can I trust the data? What patterns or stories does it contain? Quantitative summaries and visualizations complement each other; association does not automatically establish causation.

2. Review of Chapter 2 — Understanding the Dataset

The observational unit determines what a row represents. Data dictionaries and metadata clarify definitions, units, and collection procedures. Storage types differ from analytical types; measurement scales affect which summaries are meaningful; granularity determines detail; and the data-generating process affects generalizability.

3. What is data quality?

Data quality is the degree to which data is suitable for a particular purpose. The same data may be sufficient for broad age-group analysis but insufficient for a calculation requiring exact age on a specific date.

4. Dimensions of data quality

Dimension	Question	Example
Completeness	Are required values and records present?	20% of income values are missing.
Validity	Do values follow defined rules?	Negative quantity where only positive sales are allowed.
Consistency	Do values and conventions agree?	Same city recorded with inconsistent labels.
Uniqueness	Are records unique where required?	A transaction is recorded twice.
Accuracy	Do values represent reality correctly?	Actual age is 53 but recorded age is 35.
Timeliness	Is the data current enough?	Inventory data is weeks old when current stock is needed.

A value can be valid but inaccurate. A plausible age of 35 may pass a range check while still being wrong. An implausible value flags a problem for investigation but does not prove the cause.

Quality is not the same as cleanliness: a dataset with no missing values can still contain incorrect measurements, biased sampling, outdated records, or inconsistent definitions.

5. Missing values

Missingness may appear as NaN, None, NULL, empty cells, empty strings, text markers such as 'Unknown', or sentinel codes such as -1 or 999. These are not automatically equivalent. Check documentation before interpreting a value as missing.

```
missing_count = df.isna().sum()\nmissing_percent = df.isna().mean() * 100\n\nmissing_summary = pd.DataFrame({\n    "
```

If 30 of 200 income values are missing, the missing percentage is 30 / 200 × 100 = 15%. This describes the observed data, not why the values are missing.

5.1 Row-level missingness and patterns

```
missing_per_row = df.isna().sum(axis=1)\nprint(missing_per_row.value_counts().sort_index())\n\nrows_with_missing = d
```

The missing-cell percentage and percentage of rows affected are different. Column-level reports can also hide missingness concentrated in groups or combinations of variables.

```
missing_flags = df.isna().astype(int)\nprint(missing_flags.corr())
```

Correlations among missingness indicators can reveal variables whose missingness tends to co-occur. They do not establish the cause and can be uninformative when a field has almost no missingness.

5.2 MCAR, MAR, and MNAR

Mechanism	Meaning	Illustrative example
MCAR	Missingness does not depend on observed or unobserved data.	Random system failure removes records without favoring particular customers.
MAR	After accounting for relevant observed variables, missingness does not depend on the missing value itself.	Income is more often unreported among younger customers, and age is observed.
MNAR	Missingness depends on the missing value itself or on information that remains unobserved.	Very high-income customers are especially likely to withhold income.

These are assumptions about how missingness arises. They generally cannot be identified conclusively from the observed table alone. Missingness may distort distributions, group comparisons, and relationships.

6. Duplicate records and keys

Exact duplicates repeat the same values across selected columns. A duplicate key occurs when a field or combination expected to be unique repeats. Repeated entity identifiers may be legitimate when one entity can have multiple records.

```
print("Exact duplicate rows:", df.duplicated().sum())\nprint(df[df.duplicated(keep=False)])\n\n# Example: inspect rep
```

Pandas marks later occurrences as duplicates by default; keep=False marks every member of a duplicate group. The example requires a Transaction ID column. Adapt it to your dataset.

6.1 Composite keys

```
key_columns = ["Store ID", "Date", "Product ID"]\nrepeated_keys = df.duplicated(subset=key_columns, keep=False)\npri
```

This assumes each Store ID–Date–Product ID combination should be unique. Establish that rule before interpreting repetitions as errors. Never remove records merely because an entity ID repeats.

7. Invalid values and domain constraints

A domain constraint defines permitted or expected values. Examples include non-negative quantities in a completed-sales table, allowed category lists, or dates within a known observation period. Rules depend on the data's meaning and business process.

```
invalid_age_mask = (\n    df["Age"].notna()\n    & ~df["Age"].between(0, 120)\n)\nprint(df.loc[invalid_age_mask])
```

The range 0–120 is an illustrative screening rule for a conventional human-age field, not a universal law. Failed checks may indicate entry errors, unit issues, sentinel codes, parsing problems, or unusual legitimate cases.

```
allowed_occupations = {\n    "Engineer", "Teacher", "Designer", "Doctor", "Other"\n}\ninvalid_mask = (\n    df["Occu
```

Use a controlled list only if it is authoritative and complete. Otherwise, an unfamiliar category may be valid.

8. Inconsistent categories and formatting

Common problems include inconsistent capitalization, extra whitespace, abbreviations, spelling variants, and old versus new category names. Inspect exact values before deciding whether they refer to the same category.

```
print(df["City"].value_counts(dropna=False))\n\nfor value in df["City"].dropna().unique():\n    print(repr(value))\n
```

Normalization here is diagnostic only. Do not silently replace original values; verify equivalence and document any approved mapping.

9. Cross-column and logical consistency

Some issues appear only when columns are considered together: a cancelled order with shipped quantity, a purchase date before signup date, or a subtotal inconsistent with quantity and unit price. Such checks require explicit business rules.

```
df["Expected Subtotal"] = df["Quantity"] * df["Unit Price"]\ndf["Subtotal Difference"] = (\n    df["Subtotal"] - df[
```

This assumes subtotal equals quantity times unit price. Discounts, taxes, returns, currency conversion, and rounding may explain discrepancies. The 0.01 tolerance is illustrative and should match business rules.

```
df["Signup Date"] = pd.to_datetime(\n    df["Signup Date"], errors="coerce"\n)\ndf["Purchase Date"] = pd.to_datetime
```

Unparseable dates become NaT in this example. Ambiguous date formats can also be misinterpreted; use documented formats and inspect failed parses. Confirm that a purchase must occur after signup before treating a conflict as an error.

10. Quantifying quality

Metric	Formula / definition
Missing-cell rate	Missing cells ÷ total cells × 100
Record missingness rate	Rows with ≥1 missing field ÷ total rows × 100
Invalid-value rate	Values violating a defined rule ÷ assessed non-missing values × 100
Duplicate-row rate	Rows marked as duplicates ÷ total rows × 100

Duplicate metrics vary: duplicated() excludes the first occurrence in each group by default. Another metric may count all rows participating in duplicate groups or count duplicate keys. State the definition and denominator. Quality categories may overlap, so do not add their percentages unless categories are mutually exclusive.

11. Reusable quality audit

This intentionally imperfect customer dataset contains missing values, inconsistent city labels, a repeated row, an implausible age, and an unusual income. It is designed for practice; in real work, the cause of anomalies is usually unknown until investigated.

```
import pandas as pd\nimport numpy as np\n\ndf = pd.DataFrame({\n    "Customer ID": ["C101", "C102", "C103", "C104",\n
```

11.1 Structure, missingness, and duplicates

```
print("Shape:", df.shape)\ndf.info()\nprint(df.head())\n\nmissing_report = pd.DataFrame({\n    "missing_count": df.is
```

11.2 Quality metrics summary

```
total_rows = len(df)\ntotal_cells = df.size\ninvalid_age_mask = (\n    df["Age"].notna() & ~df["Age"].between(0, 120
```

This is a first-pass report, not a complete quality assessment. It cannot establish accuracy, representativeness, or the cause of each anomaly.

12. Professional workflow

Step	Action
1. Establish context	Confirm purpose, observational unit, definitions, and expected rules.
2. Profile	Inspect types, missingness, duplicates, categories, and ranges.
3. Investigate	Check source records, domain rules, related columns, and collection procedures.
4. Document	Separate observations, rules, interpretations, and unresolved questions.
5. Communicate	Explain potential impact and what requires clarification.

For each issue, record the evidence, the rule or expectation, the interpretation, and the next step. Prioritize by scope, importance, risk of bias, uncertainty, and feasibility of resolution.

13. Common mistakes

• Treating every missing value as an error.

• Assuming all duplicate records are mistakes.

• Declaring every extreme value invalid.

• Using thresholds without justification.

• Confusing validity with accuracy.

• Treating normalization as proof of equivalence.

• Ignoring cross-column contradictions.

• Reporting metrics without definitions or denominators.

• Silently modifying source data during EDA.

• Overstating what automated checks prove.

14. Chapter summary

Assess data quality in context. Quantify missingness, inspect patterns, distinguish repeated entities from duplicate records, apply justified domain constraints, check category and cross-column consistency, and document findings with appropriate uncertainty. Detection is not the same as diagnosis or repair.

Central principle: detecting a data quality issue is not the same as knowing its cause or knowing how it should be fixed.

15. Assignment

Part A — Conceptual questions

• Distinguish validity and accuracy.

• Why does quality depend on analytical purpose?

• Distinguish missing-cell percentage from the percentage of rows affected.

• Explain MCAR, MAR, and MNAR with examples.

• Why may repeated customer IDs be valid in transaction data?

• How can a value pass a range check and still be inaccurate?

• Why investigate rather than automatically remove unusual values?

• Distinguish exact duplicate rows and duplicate keys.

• Why should quality metrics define denominators?

• What belongs in a professional quality report?

Part B — Practical investigation

Use the imperfect customer dataset. Calculate column-level missingness, percentage of rows affected, exact duplicates, repeated customer IDs, and ages outside the illustrative range 0–120. Investigate city-label variations, calculate appropriate issue rates, and create a table with each issue, evidence, potential impact, and next investigation step. Explain which findings are confirmed violations and which remain suspicious.

Do not delete duplicates, fill missing values, normalize original categories, or remove suspicious observations. This is an assessment exercise, not a data-repair exercise.

16. Coding project — Reusable Data Quality Audit

Extend your cumulative EDA master notebook with a reusable quality-assessment section: dataset context; missingness counts and percentages; row-level missingness; exact duplicates and configurable key checks; domain checks; category diagnostics; cross-column checks where rules are known; a quality metrics summary; and a written report of findings and uncertainty.

Keep original values unchanged; define every rule; handle missing values explicitly; separate confirmed violations from potential issues; and identify which columns and rules need configuration.

17. Preview of Next Chapter

Chapter 4 — Exploring Numerical Variables

Next, examine one numerical variable at a time using counts, means, medians, quantiles, ranges, variance, standard deviation, and other descriptive statistics. Investigate how summaries can mislead when distributions are skewed, values

are unusual, or samples are limited.